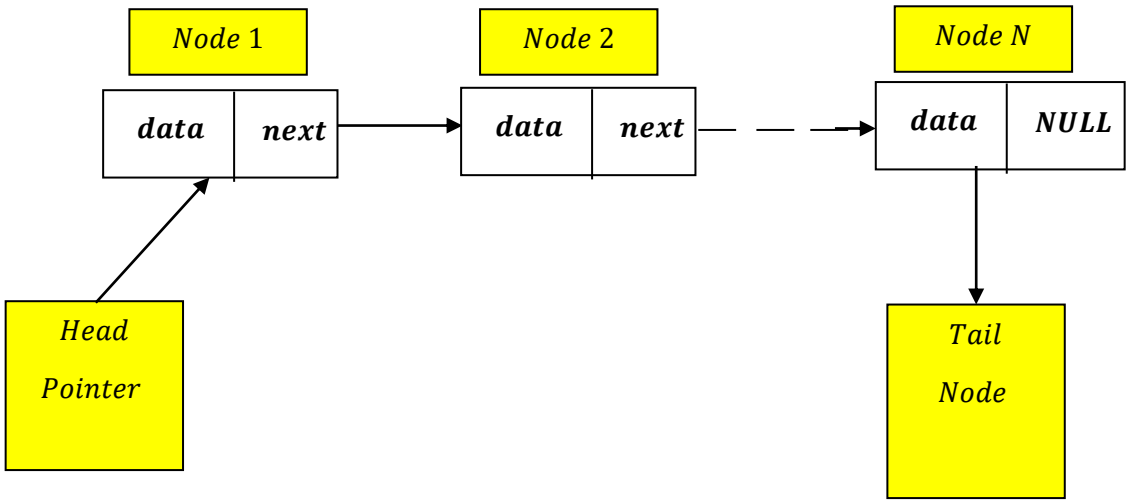
1. *Singly Linked List – Introduction*

The linked list consist of a series of structures called nodes. We can think of each node as a record. The first part of the record is a field that stores the data, and the second part of the record is a field that stores a pointer to a node. So, each node contains two fields: ``a data field`` and ``a next field`` which is a pointer used to link one node to the next node.

Generally ``linked list`` means a singly linked list. This list consists of a number of nodes in which each node has a next pointer to the following element. The link of the last node in the list is NULL, which indicates the end of list.

Each node is allocated in the heap with a call to malloc(), so the node memory continues to exist until it is explicitly deallocated with a call to free().

The node called a head is the first node in the list. The last node's next pointer points to NULL value.



A linked list or one way list , is a linear collection of data elements called nodes, where the linear order is given by means of pointers. That is each node is divided into two parts: the first part contains the information of the element, and the second part called the link field or next pointer field, contains the address of the next node in the list.

→ *A singly linked list or simply a linked list can be visualized as a list of zero or more elements of the same type.*

→ *A list having zero elements is called a null list.*

→ *Each element is stored as a node.*

→ *In a singly linked list, a node has an item or element portion where any data can be stored.*

→ *The node also has a link that points to the next node of the list.*

→ *A link or pointer indicates the address of the next node.*

→ *The first node is pointed to by a special pointer called the head, which stores its address.*

→ *This head pointer is very crucial as the elements of a list can be accessed only through this pointer using a process called chaining.*

→ *The last node is called the tail.*

→ *The tail node is indicated by a grounded pointer called a null pointer (also called a nil pointer).*

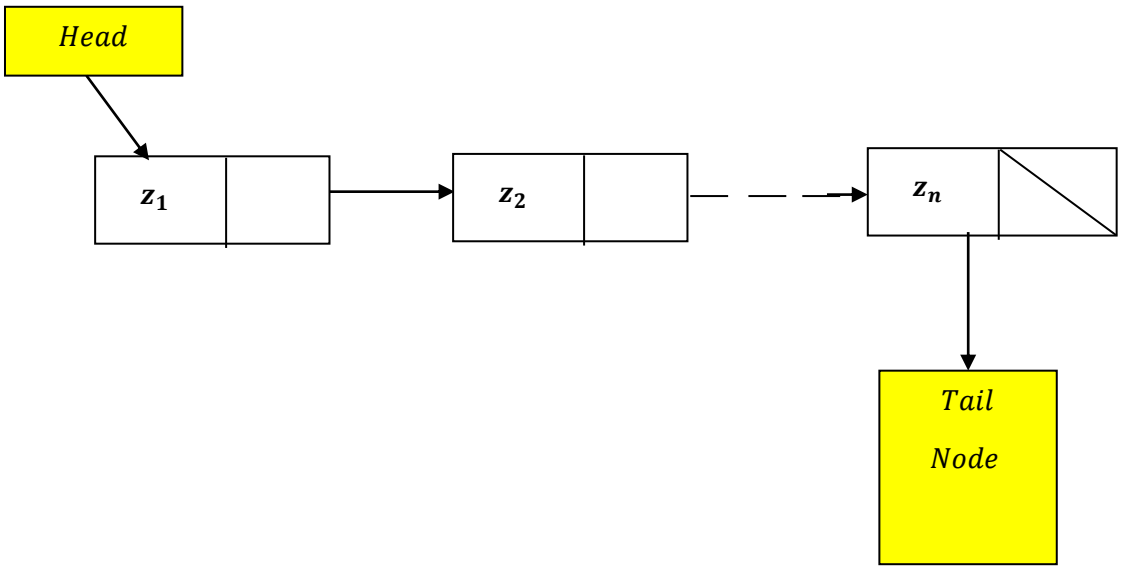
→ *The tail node marks the end of a linked list.*

→ *The number of elements of a linked list is called the size of the list.*

→ *If the size of a linked list is zero , then the linked list is called a null list.*

→ *Elements of a linked list are ordered linearly, say from* 1 *to N. If so , the elemnt* z_i *is said to be the predecessor of the element* z_{i+1}.

→ *The* z_{i+1} *can also be called a successor of the element* z_i.



→ *It can be observed that the first element has no predecessor and the last element has no successor.*

→ *The first node of a linked list is pointed at by a special pointer called the head.*

→ *The last element of the list , has no successor and is called a tail node. This is indicated by a grounded or null pointer.*

Differences between an array and a linked list

S.no.	*Array*	*Linked List*
1.	*Random access is possible.*	*Random access is not possible and one has to go through all the nodes before reaching the desired node.*
2.	*Insertion and deletion of an element require data movement.*	*Much data movement is not required.*
3.	*Size of the array is fixed.*	*Size is not fixed.*
4.	*Elements need to be stored contiguously.*	*Data need not to be stored contiguously.*
5.	*Memory allocation should be done in the beginning, and more space is wasted.*	*Memory allocation is based on demand, and hence there is no wastage of memory.*